

Mestrado em Engenharia e Ciência de Dados

Visualização de Dados e Informação

Ficha de exercícios 3 – A caminho da recolha de dados em Digital Twin

1. Pretende-se analisar, explorar e visualizar o dataset “landmarks_pose.csv” gerado na aula através do conjunto de vídeos do **Handball Action Dataset**, disponível em <https://www.kaggle.com/datasets/mexwell/handball-action-dataset>.
 - a. Abstração de dados → inspecione o arquivo csv e responda às seguintes questões:
 - i. Qual é o tipo do dataset?
 - ii. Quantos atributos tem?
 - iii. De que tipo são os atributos (cada um deles)?
 - iv. O que significa cada um deles (consultar fonte, se necessário)?
 - v. Quantos itens tem dataset?
 - b. Faça o seguinte conjunto de testes recorrendo à linguagem de programação Python e à biblioteca Pandas.

```
import pandas as pd
import numpy as np
from pathlib import Path

path = Path("Landmarks_pose.csv") # ajusta se necessário
df = pd.read_csv(path)

print(df.head())

print(df.info())

print(df.shape)

print(df.describe())

print(df.info())
```

- i. Interprete as saídas apresentadas pela consola e compare os resultados (que fizerem sentido comparar) com as respostas dadas na alínea anterior.

- c. Para ver a estatística das frames completas em termos de golo e defesa, corra o seguinte código:

```
df = pd.read_csv("Landmarks_pose.csv")

def normalize_result(x):
    if pd.isna(x): return np.nan
    return str(x).strip().lower()

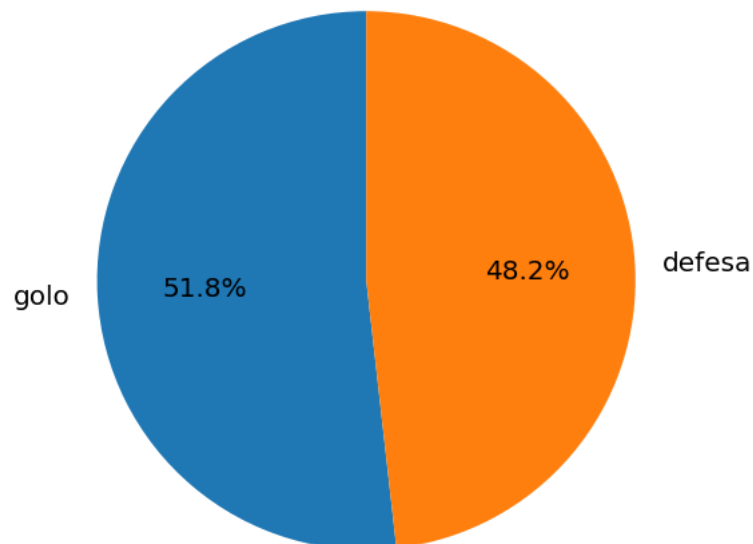
series = df["resultado"].map(normalize_result)
wanted = series.value_counts().reindex(["golo", "defesa"]).fillna(0).astype(int)

# 3) Tabela resumo e pie chart único
total = int(wanted.sum())
summary = wanted.to_frame("count")
summary["percent"] = (summary["count"] / total * 100).round(2) if total>0 else 0.0
print(summary)

plt.figure()
plt.pie(wanted.values, labels=wanted.index.tolist(), autopct="%1.1f%%", startangle=90)
plt.title("Distribuição total de frames adquiridas: golo vs defesa")
plt.axis("equal")
plt.show()
```

Eis o que deve surgir em gráfico:

Distribuição total de frames adquiridas: golo vs defesa



Olhando para o gráfico, o que é que pode ser concluído da informação apresentada? Faça a análise em termos de número de vídeos (guarda-redes) inspecionados e balanceamento dos tempos de gravação ou número de frames recolhidos por vídeo onde ocorreu “golo” vs. vídeo onde ocorreu “defesa”.

- d. Agora, para agrupar o movimento das pernas e braços relativamente à posição da cabeça e considerando ainda etiquetas de categorização “golo”/”defesa”, implemente o seguinte código:
- i. Para definir os primeiros métodos auxiliares (determinação da taxa de “golo”/”defesa”), o carregamento do csv “landmarks_pose.csv” e rótulos para preparar para ler dados de pernas e braços:

```
# Converter para flag 1/0 para agregação (média = taxa de golo)
def to_goal_flag(x):
    if pd.isna(x): return np.nan
    return 1 if x == "golo" else (0 if x == "defesa" else np.nan)

def has_xy(prefix):
    return f"{prefix}_x" in df.columns and f"{prefix}_y" in df.columns

df = pd.read_csv("landmarks_pose.csv")

VIS_THR = 0.5 # limiar de visibility (se existir a coluna *_vis)
TIME_COL_CANDIDATES = ["timestamp_s", "time_s", "t", "timestamp"]

LEGS = ["perna_dir_anca", "perna_dir Joelho", "perna_dir tornoz",
        "perna_esq_anca", "perna_esq Joelho", "perna_esq tornoz"]
ARMS = ["braco_dir ombro", "braco_dir cotov", "braco_dir pulso",
        "braco_esq ombro", "braco_esq cotov", "braco_esq pulso"]

df["_resultado_norm"] = df["resultado"].map(normalize_result)
df["_goal_flag"] = df["_resultado_norm"].map(to_goal_flag)

missing = [p for p in ["cabeça_0"]+LEGS+ARMS if not has_xy(p)]
if missing:
    raise ValueError(f"Faltam colunas _x/_y para: {missing}")
```

- ii. Métodos para recolher dados acerca da visibilidade dos membros e calcular o RMSE dos mesmos relativamente à *landmark* da cabeça.

```

def vis_ok(series_name, g):
    """Máscara de visibilidade >= VIS_THR, se existir *_vis; caso contrário, tudo True."""
    col = f"{series_name}_vis"
    if col in g.columns:
        return g[col] >= VIS_THR
    return pd.Series(True, index=g.index)

# =====
# RMSE de movimento relativo
# =====
def rmse_relative_motion(group_df, points, head="cabeca_0"):
    """
    RMSE dos deslocamentos frame-a-frame do vetor relativo (ponto - cabeça),
    em 2D (x,y), com média sobre os pontos fornecidos.
    """
    # ordenar por tempo se existir coluna temporal
    time_col = "timestamp_s"
    g = group_df.sort_values(time_col) if time_col else group_df.sort_index()

    hx = g[f"{head}_x"]; hy = g[f"{head}_y"]
    mask_h = vis_ok(head, g)

    rmses = []
    for p in points:
        px = g[f"{p}_x"]; py = g[f"{p}_y"]
        mask_p = vis_ok(p, g)
        valid = (mask_h & mask_p).values
        if valid.sum() < 3:
            continue

        # vetores relativos
        rx = (px - hx).values[valid]
        ry = (py - hy).values[valid]

        # deltas frame-a-frame
        dx = np.diff(rx)
        dy = np.diff(ry)
        if dx.size == 0:
            continue

        rmse = np.sqrt(np.mean(dx*dx + dy*dy))
        rmses.append(rmse)

    return float(np.mean(rmses)) if rmses else np.nan

```

- iii. Agregar dados por do RMSE pernas e braços por jogador.

```

records = []
for gk, gdf in df.groupby("idx_gk"):
    legs_rmse = rmse_relative_motion(gdf, LEGS, "cabeca_0")
    arms_rmse = rmse_relative_motion(gdf, ARMS, "cabeca_0")

    # decisão golo/defesa por GK (maioria/mean da flag nos frames com valor)
    flags = gdf["_goal_flag"].dropna().astype(float)
    if len(flags) == 0:
        label_text = "n/a"
    else:
        label_text = "golo" if flags.mean() >= 0.5 else "defesa"

    records.append({
        "idx_gk": gk,
        "pernas_rmse": legs_rmse,
        "bracos_rmse": arms_rmse,
        "Label": label_text
    })

summary = pd.DataFrame.from_records(records).sort_values("idx_gk").reset_index(drop=True)
print(summary)

```

- iv. E, finalmente, realizar as configurações necessárias para reproduzir um gráfico de pares de barras onde é indicado, por guarda-redes, o seu grau de movimentação.

```

labels = summary["idx_gk"].astype(str).tolist()
legs_vals = summary["pernas_rmse"].to_numpy()
arms_vals = summary["bracos_rmse"].to_numpy()
gk_outcome = summary["Label"].tolist()

x = np.arange(len(labels))
width = 0.4

plt.figure(figsize=(max(6, len(labels)*1.2), 5))
b1 = plt.bar(x - width/2, legs_vals, width, label="pernas")
b2 = plt.bar(x + width/2, arms_vals, width, label="bracos")

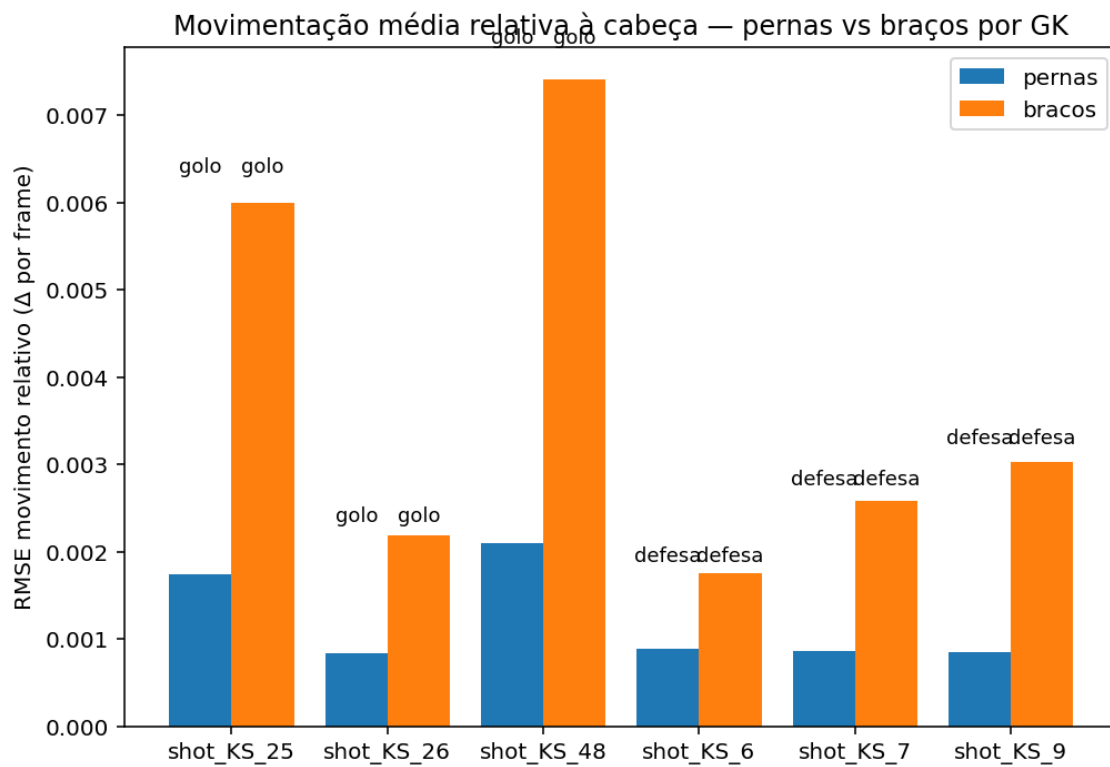
plt.xticks(x, labels)
plt.ylabel("RMSE movimento relativo ( $\Delta$  por frame)")
plt.title("Movimentação média relativa à cabeça – pernas vs braços por GK")
plt.legend()

# Anotar golo/defesa por GK acima de cada par de barras
for i, txt in enumerate(gk_outcome):
    # posição acima das barras (pega no topo do mais alto entre as duas)
    top = np.nanmax([legs_vals[i], arms_vals[i]])
    if np.isnan(top):
        top = 0.0
    y_annot = top * 1.05 if top > 0 else 0.01
    plt.text(x[i] - width/2, y_annot, txt, ha="center", va="bottom", fontsize=9, rotation=0)
    plt.text(x[i] + width/2, y_annot, txt, ha="center", va="bottom", fontsize=9, rotation=0)

plt.tight_layout()
plt.show()

```

O resultado esperado é este:



Analise o gráfico e procure retirar ilações decorrentes da comparação dos RMSEs com a confirmação de “golo”/”defesa”.